EXPERIMENT – 01

AIM:

The lexical analyzer should ignore redundant spaces, tabs, and new lines. It should also ignore comments. Although the syntax specification states that identifiers can be arbitrarily long, you may restrict the length to some reasonable value. Develop a lexical Analyzer to identify identifiers, constants, and operators using the C program.

ALGORITHM:

Step 1: Start the program.

Step 2: Read the input C program character by character.

Step 3: Ignore redundant spaces, tabs, and newlines.

Step 4: Ignore comments beginning with // or enclosed between /* and */.

Step 5: Identify and display tokens as follows:

- Identifiers: Start with a letter or underscore, followed by letters, digits, or underscores.
- Constants: Identify integer and floating-point constants.
- Operators: Identify arithmetic, relational, logical, assignment, and other operators.

Step 6: Ignore other characters that are not part of the required token categories.

Step 7: Repeat until the end of the input is reached.

Step 8: Stop the program.

CODE:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#define MAX 100
int main() {
   char s[1000], token[MAX];
   int i = 0, j, k;
   printf("Enter a C statement:\n");
   fgets(s, sizeof(s), stdin);
   while (s[i] != '\0') {
      if (isspace((unsigned char)s[i])) {
         i++;
```

```
}
else if (s[i] == '/' && s[i + 1] == '/') {
    break;
}
else if (s[i] == '/' && s[i + 1] == '*') {
    i += 2;
    while (s[i] != '\0' &&
           !(s[i] == '*' && s[i + 1] == '/'))
        i++;
    if (s[i] != '\0')
        i += 2;
}
else if (isalpha((unsigned char)s[i]) || s[i] == '_') {
    j = 0;
    while (isalnum((unsigned char)s[i]) || s[i] == '_') {
        if (j < MAX - 1)
            token[j++] = s[i];
        i++;
    }
    token[j] = '\0';
    printf("Identifier: %s\n", token);
}
else if (isdigit((unsigned char)s[i])) {
    j = 0;
    while (isdigit((unsigned char)s[i]) || s[i] == '.') {
        if (j < MAX - 1)
            token[j++] = s[i];
        i++;
    }
    token[j] = '\0';
```

```
            printf("Constant: %s\n", token);
        }
        else {
            char op[3] = {s[i], '\0', '\0'};

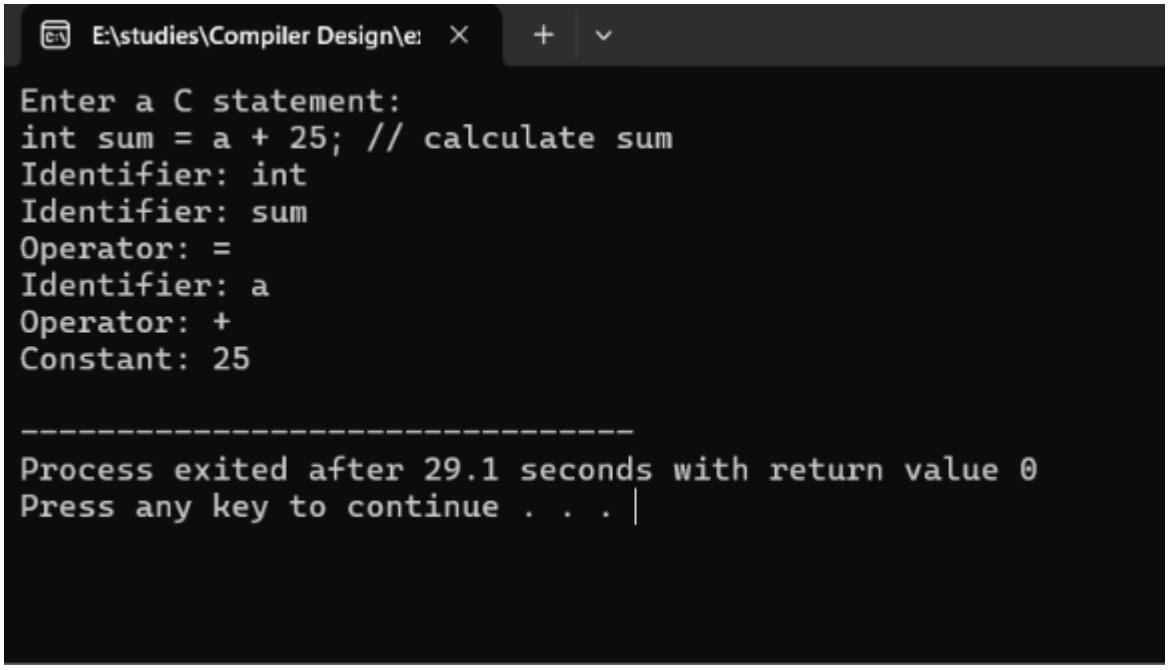
            if (strchr("+-*/%=!<>&|^", s[i])) {
                if ((s[i + 1] == '=' &&
                     strchr("+-*/%=!<>", s[i]) != NULL) ||
                    (s[i] == '+' && s[i + 1] == '+') ||
                    (s[i] == '-' && s[i + 1] == '-') ||
                    (s[i] == '&' && s[i + 1] == '&') ||
                    (s[i] == '|' && s[i + 1] == '|')) {
                    op[1] = s[i + 1];
                    i++;
                }
                printf("Operator: %s\n", op);
            }
            i++;
        }
    }
    return 0;
}
```

Input:

Enter a C statement:

int sum = a + 25; // calculate sum

Output:



Result

Thus, the C program implements a simple lexical analyzer that identifies identifiers, constants, and operators while ignoring whitespace and single-line or multi-line comments.

EXPERIMENT - 02

AIM:

Extend the lexical Analyzer to Check comments, defined as follows in C:

a) A comment begins with // and includes all characters until the end of that line.
b) A comment begins with /* and includes all characters through the next occurrence of the character sequence */Develop a lexical Analyzer to identify whether a given line is a comment or not.

Algorithm:

1. Start the program.
2. Read a line of input from the user.
3. Check whether the line begins with //.
    - If yes, it is a single-line comment.
4. Otherwise, check whether the line begins with /* and contains */ after it.
    - If yes, it is a multi-line comment.
5. If neither condition is satisfied, it is not a comment.
6. Display the result.
7. Stop the program.

Code:

```
#include <stdio.h>

#include <string.h>

int main() {

   char line[500];

   int len, i, isComment = 0;

   printf("Enter a line:\n");

   fgets(line, sizeof(line), stdin);



   len = strlen(line);
```

```
// Remove trailing newline
if (len > 0 && line[len - 1] == '\n')
    line[--len] = '\0';

// Check single-line comment
if (len >= 2 && line[0] == '/' && line[1] == '/') {
    isComment = 1;
}
// Check multi-line comment
else if (len >= 4 && line[0] == '/' && line[1] == '*') {
    for (i = 2; i < len - 1; i++) {
        if (line[i] == '*' && line[i + 1] == '/') {
            isComment = 1;
            break;
        }
    }
}

if (isComment)
    printf("The given line is a comment.\n");
else
    printf("The given line is not a comment.\n");

return 0;
```

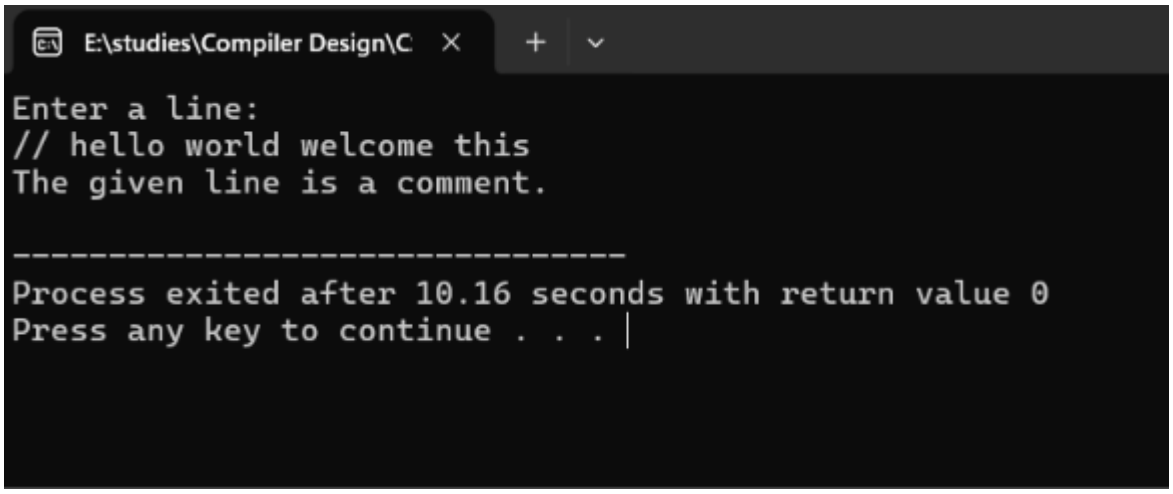
}

Input:

Enter a line:

// hello world welcome this

Output:



Result

Thus, the C program identifies whether the given line is a single-line comment, a complete multi-line comment, or not a comment. It assumes the comment marker begins at the first character of the input line.

EXPERIMENT – 03

AIM:

Design a lexical Analyzer for a given language should ignore the redundant spaces, tabs, and new lines and ignore comments.

ALGORITHM:

1. Start the program.
2. Read the input source code character by character.
3. Ignore redundant spaces, tabs, and newline characters.
4. If `//` is encountered, ignore all characters until the end of the line.
5. If `/*` is encountered, ignore all characters until the next `*/`.
6. Display the remaining characters after removing whitespace and comments.
7. Repeat until the end of the input is reached.
8. Stop the program.

CODE:

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char s[1000];
    int i = 0;

    printf("Enter the source code:\n");
    fgets(s, sizeof(s), stdin);

    printf("\nOutput after ignoring spaces and comments:\n");

    while (s[i] != '\0') {
        // Ignore spaces, tabs and newlines
        if (isspace((unsigned char)s[i])) {
            i++;
```

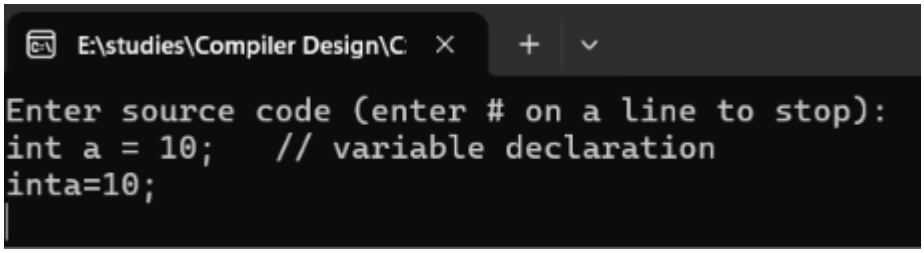
```
        }
        // Ignore single-line comments
        else if (s[i] == '/' && s[i + 1] == '/') {
            break;
        }
        // Ignore multi-line comments
        else if (s[i] == '/' && s[i + 1] == '*') {
            i += 2;
            while (s[i] != '\0' &&
                   !(s[i] == '*' && s[i + 1] == '/')) {
                i++;
            }
            if (s[i] != '\0')
                i += 2;
        }
        // Display remaining characters
        else {
            putchar(s[i]);
            i++;
        }
    }

    printf("\n");
    return 0;
}
```

INPUT:

int a = 10; // variable declaration

OUTPUT:



RESULT:

The lexical analyzer removes whitespace and comments from the input source code and displays the remaining characters.

EXPERIMENT – 04

AIM:

Design a lexical Analyzer to validate operators to recognize the operators +, -, *, / using regular Arithmetic operators.

ALGORITHM:

1. Start the program.
2. Read an arithmetic operator from the user.
3. Check whether the entered character is +, -, *, or /.
4. If it matches any of these operators, display “Valid Arithmetic Operator”.
5. Otherwise, display “Invalid Arithmetic Operator”.
6. Stop the program.

CODE:

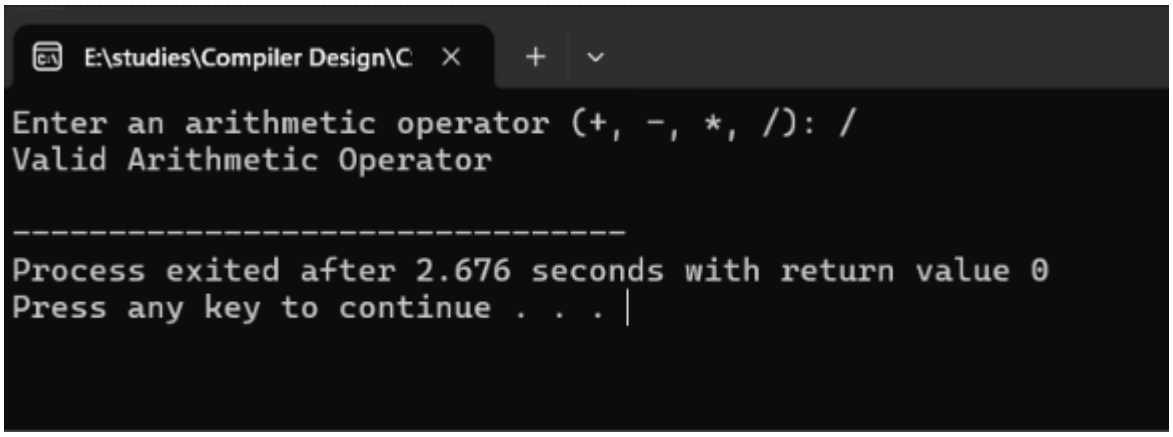
```
#include <stdio.h>
int main() {
   char op;
   printf("Enter an arithmetic operator (+, -, *, /): ");
   scanf(" %c", &op);
   if (op == '+' || op == '-' || op == '*' || op == '/') {
      printf("Valid Arithmetic Operator\n");
   } else {
      printf("Invalid Arithmetic Operator\n");
   }

   return 0;
}
```

INPUT:

Enter an arithmetic operator (+, -, *, /): /

OUTPUT:



RESULT:

Thus, the C program successfully validates arithmetic operators and recognizes +, -, *, and / as valid operators.

EXPERIMENT – 05

AIM:

Design a lexical Analyzer to find the number of whitespaces and newline characters.

ALGORITHM:

1. Start the program.
2. Read the input text character by character until EOF is reached.
3. Initialize whitespace and newline counters to zero.
4. If the character is a space or tab, increment the whitespace counter.
5. If the character is a newline (\n), increment the newline counter.
6. Display the total number of whitespaces and newline characters.
7. Stop the program.

CODE:

```
#include <stdio.h>
int main() {
   char ch;
   int whitespace = 0, newline = 0;

   printf("Enter text (press Ctrl+Z then Enter to finish):\n");

   while ((ch = getchar()) != EOF) {
      if (ch == ' ' || ch == '\t') {
         whitespace++;
      }
      if (ch == '\n') {
         newline++;
      }
   }
```

```
    printf("\nNumber of whitespaces: %d\n", whitespace);
    printf("Number of newline characters: %d\n", newline);

    return 0;
}
```

INPUT:

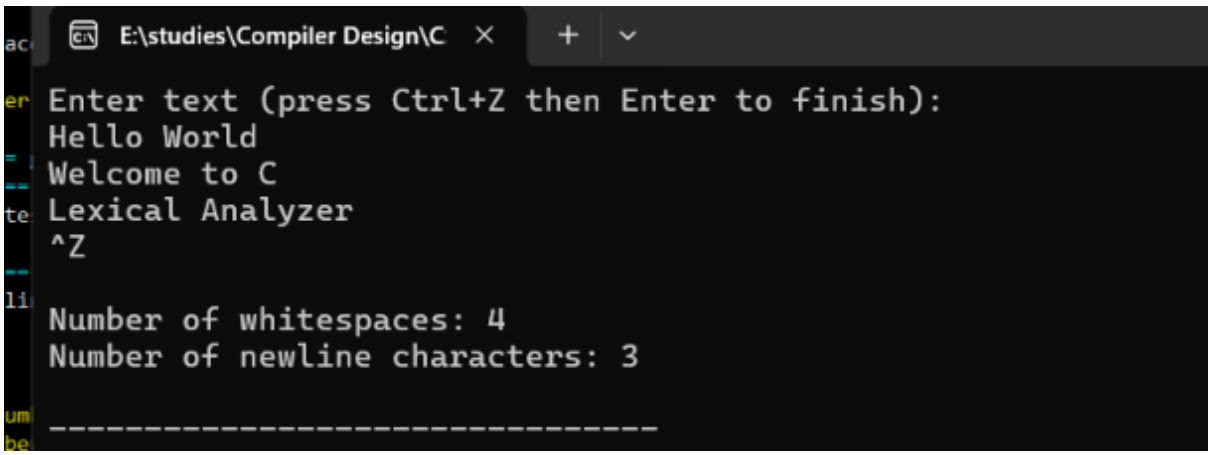
Enter text (press Ctrl+Z then Enter to finish):

Hello World

Welcome to C

Lexical Analyzer

OUTPUT:



RESULT:

Thus, the C program counts and displays the total number of whitespace characters (spaces and tabs) and newline characters in the given input.

EXPERIMENT – 06

AIM:

Develop a lexical Analyzer to test whether a given identifier is valid or not.

ALGORITHM:

1. Start the program.
2. Read an identifier from the user.
3. Check whether the first character is an alphabet or underscore (_).
4. Check whether the remaining characters contain only alphabets, digits, or underscores.
5. If both conditions are satisfied, display Valid Identifier.
6. Otherwise, display Invalid Identifier.
7. Stop the program.

CODE:

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char id[100];
    int i, valid = 1;

    printf("Enter an identifier: ");
    scanf("%99s", id);

    if (!(isalpha((unsigned char)id[0]) || id[0] == '_')) {
        valid = 0;
    } else {
        for (i = 1; id[i] != '\0'; i++) {
```

```
            if (!(isalnum((unsigned char)id[i]) || id[i] == '_')) {
                valid = 0;
                break;
            }
        }
    }

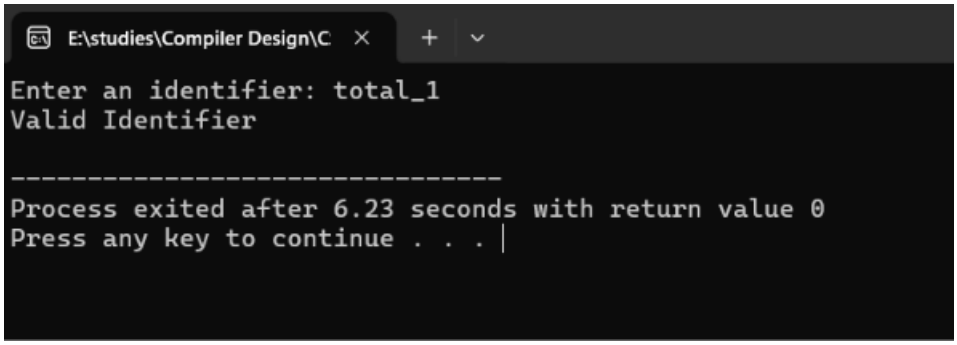
    if (valid)
        printf("Valid Identifier\n");
    else
        printf("Invalid Identifier\n");

    return 0;
}
```

INPUT:

Enter an identifier: total_1

OUTPUT:



RESULT:

Thus, the C program checks whether a given identifier follows the standard naming rules for C identifiers.

EXPERIMENT – 07

AIM:

Write a C program to find FIRST( ) - predictive parser for the given grammar

S → AaAb / BbBa

A → ∈

B → ∈

ALGORITHM:

1. Start the program.
2. Define the grammar:
    a. . S -> AaAb | BbBa
    b. . A -> E
    c. B -> E
2. Initialize the FIRST sets for non-terminals S, A, and B.
3. Since A -> E, set FIRST(A) = .
4. Since B -> €, set FIRST(B) = .
5. For S -> AaAb, since A is nullable, the first terminal is a . Thus, FIRST(AaAb) =.
6. For S -> BbBa, since B is nullable, the first terminal is b . Thus, FIRST(BbBa) =.
7. Combine the results: FIRST(S) = .
8. Display the FIRST sets.
9. Stop the program.

CODE:

```
#include <stdio.h>

int main() {

   printf("Grammar:\n");

   printf("S -> AaAb | BbBa\n");

   printf("A -> epsilon\n");

   printf("B -> epsilon\n\n");


   printf("FIRST Sets:\n");

   printf("FIRST(S) = { a, b }\n");
```

```
    printf("FIRST(A) = { epsilon }\n");
    printf("FIRST(B) = { epsilon }\n");

    return 0;
}
```

INPUT:

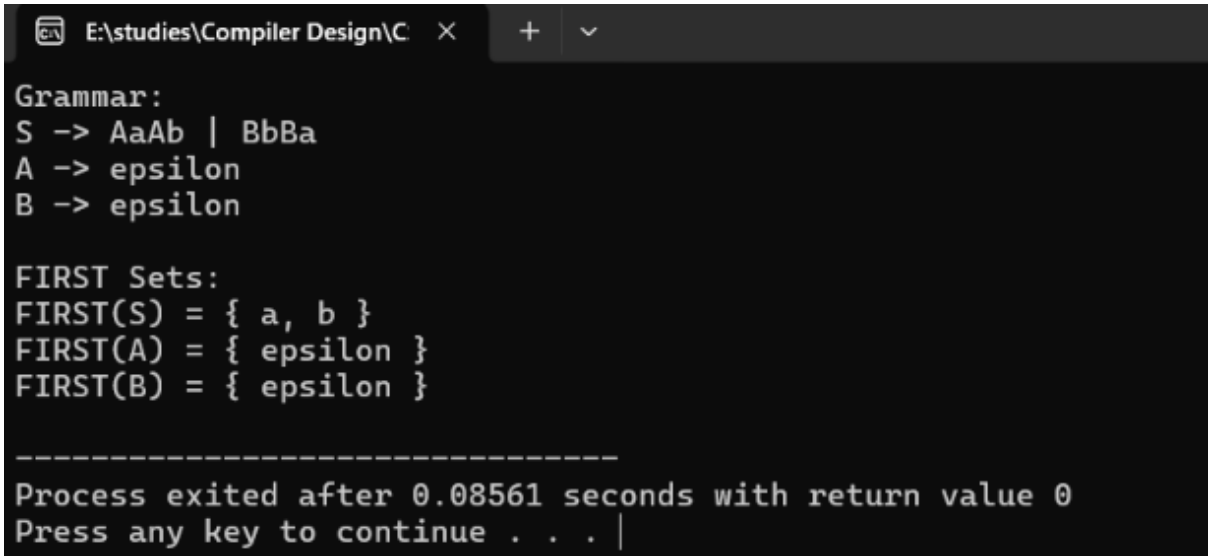
Grammar:

S -> AaAb | BbBa

A -> epsilon

B -> epsilon

OUTPUT:



RESULT:

Thus, the C program displays the FIRST sets for the given grammar:

FIRST Sets:

FIRST(S) = { a, b }

FIRST(A) = { epsilon }

FIRST(B) = { epsilon }

EXPERIMENT – 08

AIM:

Write a C program to find FOLLOW( ) - predictive parser for the given grammar

S → AaAb / BbBa

A → ∈

B → ∈

ALGORITHM:

1. Start the program.
2. Define the grammar:
    a. S -> AaAb | BbBa
    b. A-> ∈
    c. B-> ∈
3. Initialize FOLLOW(S), FOLLOW(A), and FOLLOW(B) as empty sets.
4. Since S is the start symbol, add $ to FOLLOW(S).
5. From S -> AaAb:
6. . The first A is followed by terminal a , so add a to FOLLOW(A).
7. . The second A is followed by terminal b, so add b to FOLLOW(A).
8. From S -> BbBa:
9. . The first B is followed by terminal b , so add b to FOLLOW(B).
10. . The second B is followed by terminal a , so add a to FOLLOW(B).
11. Display all FOLLOW sets.
12. Stop the program.

CODE:

```
#include <stdio.h>
int main() {
   printf("Grammar:\n");
   printf("S -> AaAb | BbBa\n");
   printf("A -> epsilon\n");
```

```
    printf("B -> epsilon\n\n");
    printf("FOLLOW Sets:\n");
    printf("FOLLOW(S) = { $ }\n");
    printf("FOLLOW(A) = { a, b }\n");
    printf("FOLLOW(B) = { a, b }\n");
    return 0;
}
```

INPUT:

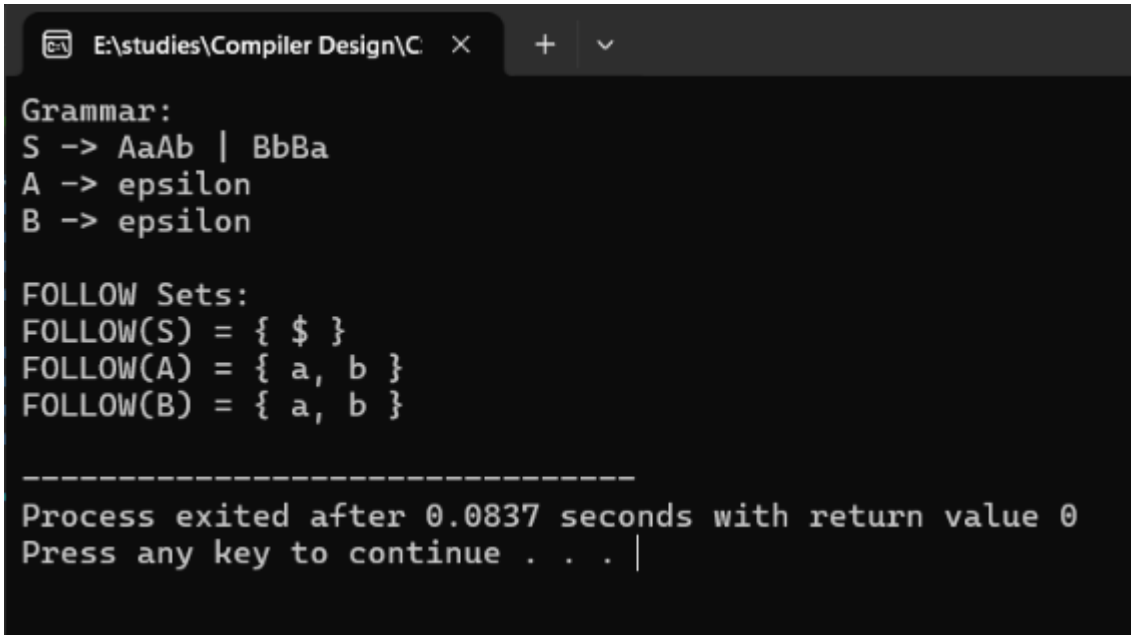
Grammer:

S -> AaAb | BbBa

A-> ∈

B-> ∈

OUTPUT:



RESULT:

Thus, the C program displays the FOLLOW sets for the given grammar:

- FOLLOW(S) = $
- FOLLOW(A) = b
- FOLLOW(B) = b

EXPERIMENT – 09

AIM:

Implement a C program to eliminate left recursion from a given CFG.

S → (L) / a

L → L , S / S

ALGORITHM:

1. Start the program.
2. Define the given grammar:
    a. S -> (L) | a
    b. L -> L, S | S
2. Identify the immediate left recursion in L -> L, S.
3. Apply the left-recursion elimination rule:
    a. If A -> Ac | B, rewrite it as:
    b. A -> BA'
    c. A' -> aA' | ∈
4. Obtain the transformed grammar
    a. S -> (L) | a
    b. . L- SL'
    c. . L' ->, SL' | €
5. Display the original and transformed grammars.
6. Stop the program.

CODE:

```
#include <stdio.h>


int main() {
   printf("Original Grammar:\n");
   printf("S -> (L) | a\n");
   printf("L -> L,S | S\n");


   printf("\nGrammar after eliminating left recursion:\n");
   printf("S  -> (L) | a\n");
```

```
    printf("L  -> SL'\n");
    printf("L' -> ,SL' | epsilon\n");


    return 0;
}
```

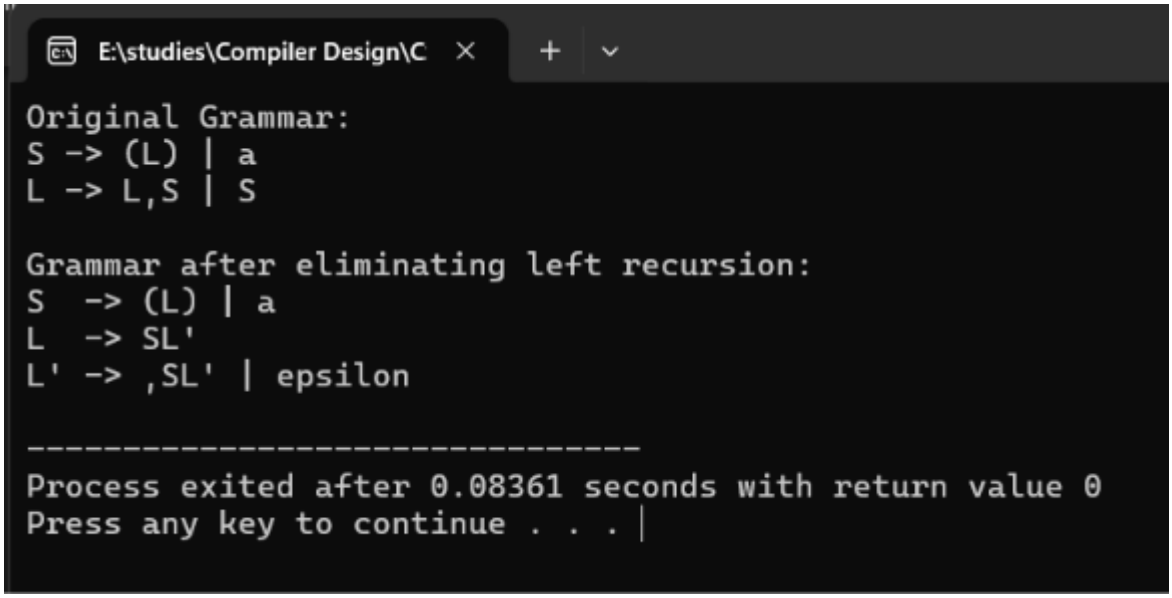
INPUT:

Original Grammar:

S -> (L) | a

L -> L,S | S

OUTPUT:



RESULT:

Thus, the C program displays the grammar after eliminating immediate left recursion.

EXPERIMENT – 10

AIM:

Implement a C program to eliminate left factoring from a given CFG.

S → iEtS / iEtSeS / a

E → b

ALGORITHM:

1. Start the program.
2. Define the given grammar:
    . S ->iEtS | iEtSeS | a
    . E->b
3. Identify the common prefix iEts in the first two productions of S.
4. Factor out the common prefix.
5. Introduce a new non-terminal S'.
6. Rewrite the grammar as:
    · S ->iEtSS' | a
    . S' ->eS | €
    . E ->b
7. Display the original and transformed grammars.
8. Stop the program.

CODE:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_RULES 20
#define MAX_ALTS 20
```

```
#define MAX_LEN 100

// Helper function to remove leading and trailing whitespace
void trim(char *str) {
    int start = 0;
    while (str[start] == ' ' || str[start] == '\t') {
        start++;
    }
    if (start > 0) {
        memmove(str, str + start, strlen(str + start) + 1);
    }
    int end = strlen(str) - 1;
    while (end >= 0 && (str[end] == ' ' || str[end] == '\t' || str[end] == '\n' ||
str[end] == '\r')) {
        str[end] = '\0';
        end--;
    }
}

// Function to eliminate left factoring from a single production rule
void eliminate_left_factoring(char *rule) {
    char lhs[MAX_LEN] = {0};
    char rhs[MAX_LEN] = {0};
    char alts[MAX_ALTS][MAX_LEN];
    int alt_count = 0;

    // Locate the production arrow '->'
    char *arrow = strstr(rule, "->");
```

```
if (!arrow) {
    printf("%s\n", rule);
    return;
}

strncpy(lhs, rule, arrow - rule);
lhs[arrow - rule] = '\0';
strcpy(rhs, arrow + 2);

trim(lhs);
trim(rhs);

// Split RHS alternatives delimited by '/' or '|'
int rhs_len = strlen(rhs);
int cur = 0;
for (int i = 0; i <= rhs_len; i++) {
    if (rhs[i] == '|' || rhs[i] == '/' || rhs[i] == '\0') {
        alts[alt_count][cur] = '\0';
        trim(alts[alt_count]);
        if (strlen(alts[alt_count]) > 0) {
            alt_count++;
        }
        cur = 0;
    } else {
        alts[alt_count][cur++] = rhs[i];
    }
}
```

```
// Rules with only 1 alternative cannot have common prefixes
if (alt_count <= 1) {
    printf("%s -> %s\n", lhs, rhs);
    return;
}
// Find the longest common prefix across all pairs of alternatives
int max_len = 0;
char common_prefix[MAX_LEN] = "";
for (int i = 0; i < alt_count; i++) {
    for (int j = i + 1; j < alt_count; j++) {
        int k = 0;
        while (alts[i][k] && alts[j][k] && alts[i][k] == alts[j][k]) {
            k++;
        }
        if (k > max_len) {
            max_len = k;
            strncpy(common_prefix, alts[i], k);
            common_prefix[k] = '\0';
        }
    }
}
// If no common prefix exists, print rule unchanged
if (max_len == 0) {
    printf("%s -> ", lhs);
    for (int i = 0; i < alt_count; i++) {
        printf("%s%s", alts[i], (i == alt_count - 1) ? "\n" : " | ");
```

```
    }
    return;
}
// Identify alternatives that share the longest prefix
int matched[MAX_ALTS] = {0};
for (int i = 0; i < alt_count; i++) {
    if (strncmp(alts[i], common_prefix, max_len) == 0) {
        matched[i] = 1;
    }
}
// 1. Output the main factored production: A -> alpha A' | gamma
printf("%s -> %s%s'", lhs, common_prefix, lhs);
for (int i = 0; i < alt_count; i++) {
    if (!matched[i]) {
        printf(" | %s", alts[i]);
    }
}
printf("\n");
// 2. Output the auxiliary production: A' -> beta1 | beta2 | ...
printf("%s' -> ", lhs);
int first = 1;
for (int i = 0; i < alt_count; i++) {
    if (matched[i]) {
        if (!first) {
            printf(" | ");
        }
        first = 0;
```

```
            char *suffix = alts[i] + max_len;
            if (strlen(suffix) == 0) {
                printf("ε"); // ε represents epsilon / empty string
            } else {
                printf("%s", suffix);
            }
        }
    }
    printf("\n");
}
int main() {
    int n;
    char rules[MAX_RULES][MAX_LEN];
    printf("Enter number of productions: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        return 0;
    }
    while (getchar() != '\n'); // Clear trailing newline

    for (int i = 0; i < n; i++) {
        printf("Enter production %d: ", i + 1);
        if (fgets(rules[i], sizeof(rules[i]), stdin)) {
            rules[i][strcspn(rules[i], "\r\n")] = '\0';
        }
    }
    printf("\n--- Grammar After Eliminating Left Factoring ---\n");
    for (int i = 0; i < n; i++) {
```

```
        eliminate_left_factoring(rules[i]);
    }
    return 0;
}
```

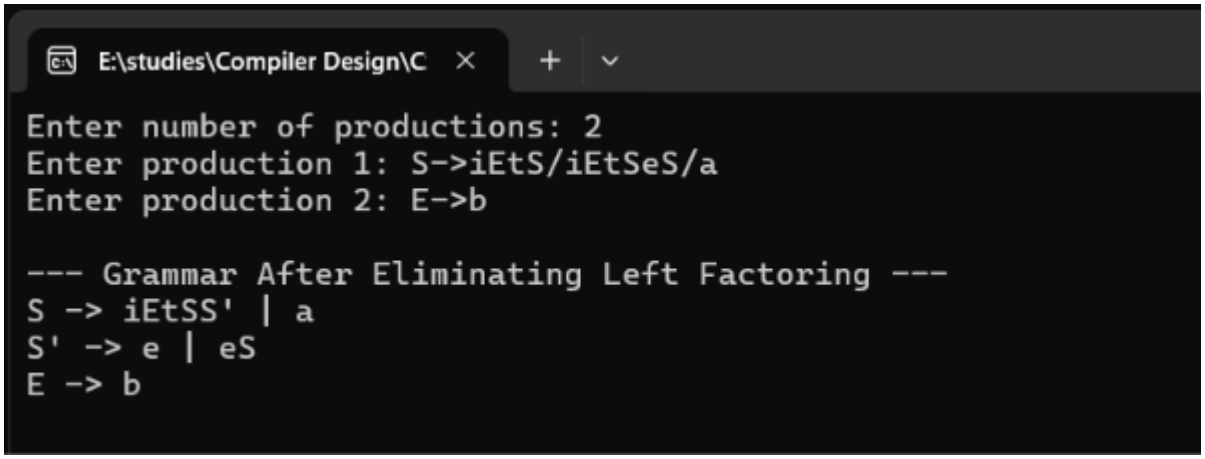
INPUT:

Enter number of productions: 2

Enter production 1: S->iEtS/iEtSeS/a

Enter production 2: E->b

OUTPUT:



RESULT:

Thus, the C program displays the left-factored grammar by removing the common prefix iEtS from the productions of S.

EXPERIMENT – 11

AIM:

Implement a C program to perform symbol table operations.

ALGORITHM:

1. Start the program.
2. Initialize an empty symbol table.
3. Display the menu:
    - Insert a symbol.
    - Display the symbol table.
    - Search for a symbol.
    - Delete a symbol.
    - Exit.
4. Perform the operation selected by the user.
5. Repeat until the user selects Exit.
6. Stop the program.

CODE:

```
#include <stdio.h>
#include <string.h>

struct Symbol {
    char name[30];
    char type[20];
    int address;
};

struct Symbol table[100];
int count = 0;

void insert() {
    char name[30], type[20];
    int i;
```

```
    printf("Enter symbol name: ");
    scanf("%29s", name);

    for (i = 0; i < count; i++) {
        if (strcmp(table[i].name, name) == 0) {
            printf("Symbol already exists!\n");
            return;
        }
    }

    printf("Enter symbol type: ");
    scanf("%19s", type);

    strcpy(table[count].name, name);
    strcpy(table[count].type, type);
    table[count].address = 1000 + count * 4;
    count++;

    printf("Symbol inserted successfully.\n");
}

void display() {
    int i;

    if (count == 0) {
        printf("Symbol table is empty.\n");
        return;
    }

    printf("\nName\tType\tAddress\n");
    printf("-----------------------------\n");

    for (i = 0; i < count; i++) {
```

```
        printf("%s\t%s\t%d\n",
               table[i].name,
               table[i].type,
               table[i].address);
    }
}

void search() {
    char name[30];
    int i;

    printf("Enter symbol to search: ");
    scanf("%29s", name);

    for (i = 0; i < count; i++) {
        if (strcmp(table[i].name, name) == 0) {
            printf("Symbol found!\n");
            printf("Name: %s\n", table[i].name);
            printf("Type: %s\n", table[i].type);
            printf("Address: %d\n", table[i].address);
            return;
        }
    }

    printf("Symbol not found.\n");
}

void deleteSymbol() {
    char name[30];
    int i, j;

    printf("Enter symbol to delete: ");
    scanf("%29s", name);
```

```
    for (i = 0; i < count; i++) {
        if (strcmp(table[i].name, name) == 0) {
            for (j = i; j < count - 1; j++) {
                table[j] = table[j + 1];
            }

            count--;
            printf("Symbol deleted successfully.\n");
            return;
        }
    }

    printf("Symbol not found.\n");
}

int main() {
    int choice;

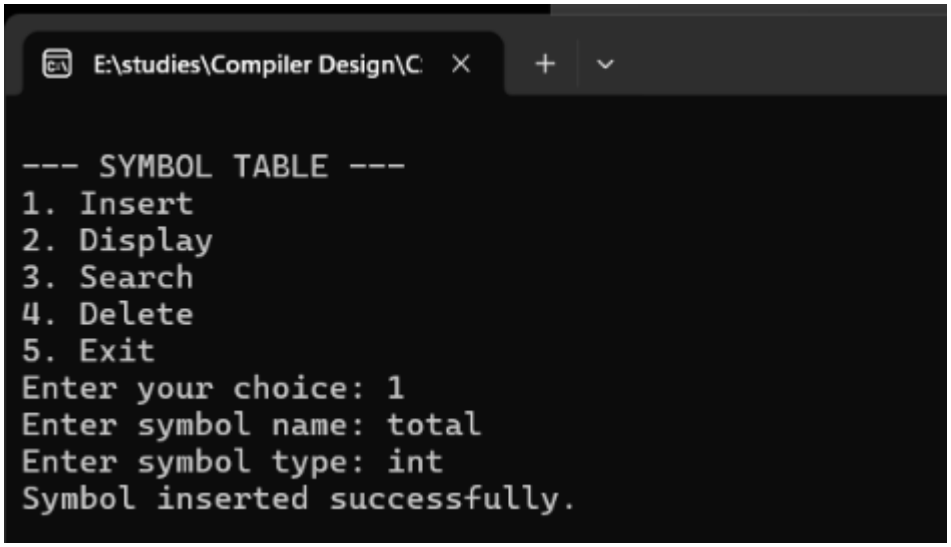
    do {
        printf("\n--- SYMBOL TABLE ---\n");
        printf("1. Insert\n");
        printf("2. Display\n");
        printf("3. Search\n");
        printf("4. Delete\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: insert(); break;
            case 2: display(); break;
            case 3: search(); break;
            case 4: deleteSymbol(); break;
            case 5: printf("Exiting program.\n"); break;
```

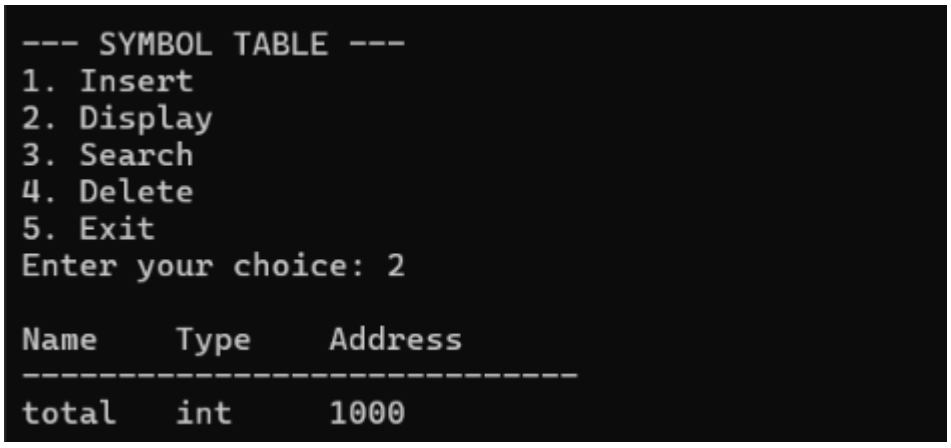
```
            default: printf("Invalid choice!\n");
        }
    } while (choice != 5);

    return 0;
}
```

INPUT:



OUTPUT:



RESULT:

Thus, the C program implements the basic symbol table operations: insertion, display, searching, and deletion.

EXPERIMENT – 12

AIM:

Write a C program to construct recursive descent parsing for the given grammar

E → TE’

E’ → +TE’ / ∈

T → FT’

T’ → *FT’ / ∈

F → (E) / id

ALGORITHM:

1. Start the program.
2. Read the input expression containing identifiers, `+`, `*`, `(`, and `)`.
3. Implement recursive functions for the grammar productions:
    - `E → TE'`
    - `E' → +TE' | ε`
    - `T → FT'`
    - `T' → *FT' | ε`
    - `F → (E) | id`
4. Call the function `E()` to begin parsing.
5. If the entire input is parsed successfully, display String Accepted.
6. Otherwise, display String Rejected.
7. Stop the program.

CODE:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char input[100];
int pos = 0;

int E();
int Eprime();
int T();
int Tprime();
```

```
int F();

int E() {
   return T() && Eprime();
}

int Eprime() {
   if (input[pos] == '+') {
      pos++;
      return T() && Eprime();
   }
   return 1;  // epsilon
}

int T() {
   return F() && Tprime();
}

int Tprime() {
   if (input[pos] == '*') {
      pos++;
      return F() && Tprime();
   }
   return 1;  // epsilon
}

int F() {
   if (input[pos] == '(') {
      pos++;
      if (E() && input[pos] == ')') {
         pos++;
         return 1;
      }
      return 0;
```

```
    }

    if (isalpha((unsigned char)input[pos])) {
        while (isalnum((unsigned char)input[pos]) ||
               input[pos] == '_') {
            pos++;
        }
        return 1;  // id
    }

    return 0;
}

int main() {
    printf("Enter an expression: ");
    scanf("%99s", input);

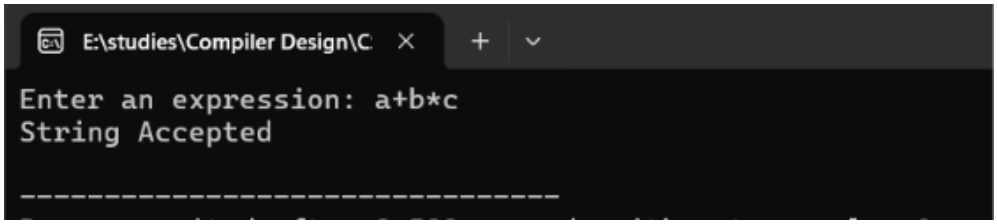
    if (E() && input[pos] == '\0')
        printf("String Accepted\n");
    else
        printf("String Rejected\n");

    return 0;
}
```

INPUT:

Enter an expression: a+b*c

OUTPUT:



RESULT: Thus, the C program implements a recursive descent parser for the given grammar and determines whether the input arithmetic expression is accepted or rejected.

EXPERIMENT – 13

AIM

To implement a C program using the Top-Down Parsing technique (Recursive Descent Parsing) to check whether a given input string satisfies the rules of the specified grammar.

ALGORITHM

1. Start the program.
2. Define the grammar productions:
    - E → TE'
    - E' → +TE' | ε
    - T → FT'
    - T' → *FT' | ε
    - F → (E) | id
3. Read the input arithmetic expression.
4. Implement recursive functions for each non-terminal: E(), Eprime(), T(), Tprime(), and F().
5. Start parsing by calling E().
6. Check whether the entire input string is consumed.
7. If parsing succeeds and the entire input is consumed, print String Accepted; otherwise, print String Rejected.
8. Stop the program.

CODE

```
#include <stdio.h>
#include <ctype.h>

char input[100];
int pos = 0;

int E();
int Eprime();
int T();
```

```
int Tprime();
int F();

int E() {
   return T() && Eprime();
}

int Eprime() {
   if (input[pos] == '+') {
      pos++;
      return T() && Eprime();
   }
   return 1;
}

int T() {
   return F() && Tprime();
}

int Tprime() {
   if (input[pos] == '*') {
      pos++;
      return F() && Tprime();
   }
   return 1;
}

int F() {
   if (input[pos] == '(') {
      pos++;
```

```
        if (E() && input[pos] == ')') {
            pos++;
            return 1;
        }
        return 0;
    }

    if (isalpha((unsigned char)input[pos])) {
        while (isalnum((unsigned char)input[pos]) ||
               input[pos] == '_') {
            pos++;
        }
        return 1;
    }

    return 0;
}

int main() {
    printf("Enter an expression: ");
    scanf("%99s", input);

    if (E() && input[pos] == '\0')
        printf("String Accepted\n");
    else
        printf("String Rejected\n");

    return 0;
}
```

INPUT

Enter an expression: a+*b

OUTPUT



RESULT

Thus, the C program successfully implements the Top-Down Parsing technique using Recursive Descent Parsing to determine whether a given input string satisfies the specified grammar.

EXPERIMENT – 14

AIM

To write a C program to generate the Three Address Code (TAC) representation for a given arithmetic expression.

ALGORITHM

1. Start the program.
2. Read an arithmetic expression from the user.
3. Scan the expression to identify operators and operands.
4. Evaluate operators according to their precedence using temporary variables.
5. Generate a three-address code statement for each operation.
6. Replace the processed subexpression with a temporary variable.
7. Repeat until the entire expression is processed.
8. Display the generated three-address code.
9. Stop the program.

CODE

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char expr[100], temp[100];
int t = 1;

int main() {
    int i, j, k, n;

    printf("Enter an expression: ");
    scanf("%99s", expr);

    n = strlen(expr);

    // Process multiplication and division first
    for (i = 0; i < n; i++) {
        if (expr[i] == '*' || expr[i] == '/') {
            char a = expr[i - 1];
            char b = expr[i + 1];

            printf("t%d = %c %c %c\n", t, a, expr[i], b);

            // Replace the subexpression with a temporary
            for (j = i - 1, k = 0; j >= 0; j--)
                temp[k++] = expr[j];
            temp[k] = '\0';

            expr[i - 1] = '0' + t;
            for (j = i; j < n - 2; j++)
```

```
                expr[j] = expr[j + 2];
            n -= 2;
            i = -1;
            t++;
        }
    }

    // Process addition and subtraction
    for (i = 0; i < n; i++) {
        if (expr[i] == '+' || expr[i] == '-') {
            char a = expr[i - 1];
            char b = expr[i + 1];

            printf("t%d = %c %c %c\n", t, a, expr[i], b);

            expr[i - 1] = '0' + t;
            for (j = i; j < n - 2; j++)
                expr[j] = expr[j + 2];
            n -= 2;
            i = -1;
            t++;
        }
    }

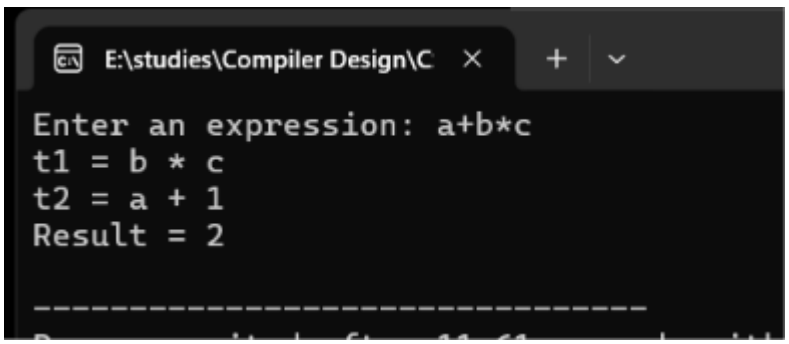
    printf("Result = %c\n", expr[0]);

    return 0;
}
```

INPUT:

```
Enter an expression: a+b*c
```

OUTPUT:



RESULT

Thus, the C program demonstrates the generation of Three Address Code (TAC) for an arithmetic expression by breaking it into simple operations using temporary variables.

EXPERIMENT - 15

AIM

To write a C program to implement a lexical analyzer that scans a file and counts the number of characters, words, and lines present in it.

ALGORITHM

1. Start the program.
2. Open the input file in read mode.
3. Initialize the character, word, and line counters to zero.
4. Read the file character by character.
5. Increment the character count for each character read.
6. Increment the word count whenever a new word begins.
7. Increment the line count whenever a newline character (`\n`) is encountered.
8. Display the total number of characters, words, and lines.
9. Close the file and stop the program.

CODE

```
#include <stdio.h>
#include <ctype.h>

int main() {
    FILE *fp;
    char filename[100];
    int ch, inWord = 0;
    int characters = 0, words = 0, lines = 0;
    int lastChar = '\0';

    printf("Enter file name: ");
    scanf("%99s", filename);

    fp = fopen(filename, "r");

    if (fp == NULL) {
        printf("Unable to open file.\n");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF) {
        characters++;
        lastChar = ch;

        if (ch == '\n')
            lines++;

        if (isspace((unsigned char)ch)) {
```

```
            inWord = 0;
        } else if (!inWord) {
            words++;
            inWord = 1;
        }
    }

    if (characters > 0 && lastChar != '\n')
        lines++;

    fclose(fp);

    printf("\nCharacters = %d\n", characters);
    printf("Words = %d\n", words);
    printf("Lines = %d\n", lines);

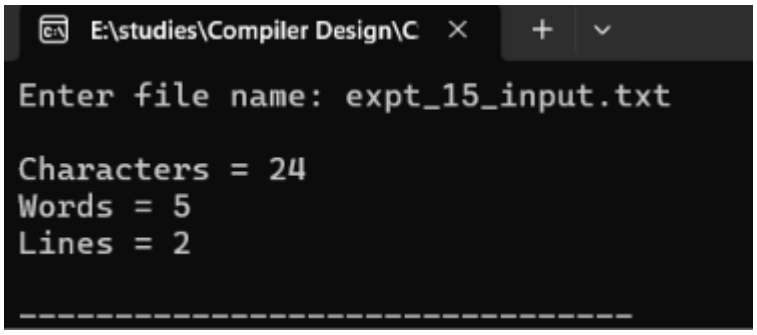
    return 0;
}
```

INPUT

`input.txt` contains:

```
Hello World
Welcome to C
```

OUTPUT



RESULT

Thus, the C program successfully scans a file and counts the total number of characters, words, and lines present in it.

EXPERIMENT - 16

AIM

To write a C program to implement the backend of a compiler by generating simple target assembly instructions from three-address code.

ALGORITHM

1. Start the program.
2. Read a three-address code statement.
3. Identify the operator, operands, and result variable.
4. Generate the corresponding assembly instructions.
5. Display the generated target code.
6. Stop the program.

CODE

```
#include <stdio.h>

int main() {
    char op, arg1[20], arg2[20], result[20];

    printf("Enter three-address code (e.g., t1=a+b): ");
    scanf("%19[^=]=%19s", result, arg2);

    // Extract the operator and operands
    if (sscanf(arg2, "%19[^+*/-]%c%19s",
               arg1, &op, arg2) == 3) {
        printf("\nTarget Assembly Code:\n");

        printf("MOV R0, %s\n", arg1);
        printf("MOV R1, %s\n", arg2);

        switch (op) {
            case '+': printf("ADD R0, R1\n"); break;
            case '-': printf("SUB R0, R1\n"); break;
            case '*': printf("MUL R0, R1\n"); break;
            case '/': printf("DIV R0, R1\n"); break;
            default:
                printf("Invalid operator\n");
                return 1;
        }

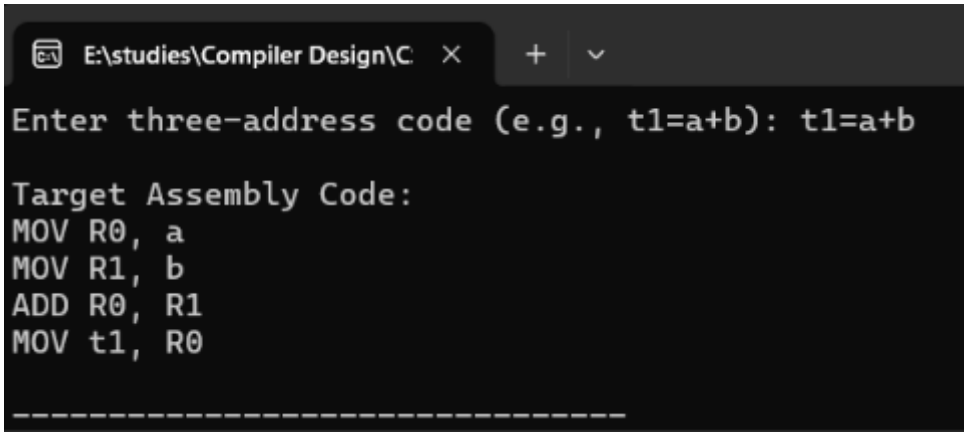
        printf("MOV %s, R0\n", result);
    } else {
        printf("Invalid three-address code\n");
    }

    return 0;
}
```

INPUT

```
Enter three-address code (e.g., t1=a+b): t1=a+b
```

OUTPUT



RESULT

Thus, the C program demonstrates the backend of a compiler by translating a simple three-address code statement into equivalent target assembly instructions.

EXPERIMENT – 17

AIM

To write a C program to compute the LEADING sets for the given grammar used in an operator precedence parser.

ALGORITHM

1. Start the program.
2. Define the given grammar:
    - `E → E + T | T`
    - `T → T * F | F`
    - `F → ( E ) | id`
3. Compute the LEADING set for each non-terminal.
4. For `E`, include `+`, `*`, `(`, and `id`.
5. For `T`, include `*`, `(`, and `id`.
6. For `F`, include `(` and `id`.
7. Display the LEADING sets.
8. Stop the program.

CODE

```
#include <stdio.h>

int main() {
    printf("GIVEN GRAMMAR:\n");
    printf("E -> E + T | T\n");
    printf("T -> T * F | F\n");
    printf("F -> ( E ) | id\n");

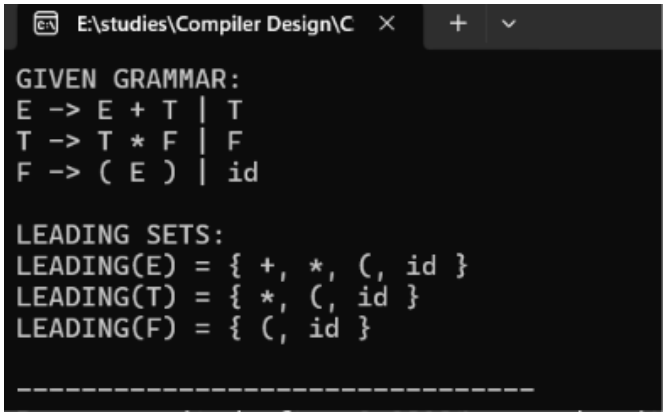
    printf("\nLEADING SETS:\n");
    printf("LEADING(E) = { +, *, (, id }\n");
    printf("LEADING(T) = { *, (, id }\n");
    printf("LEADING(F) = { (, id }\n");

    return 0;
}
```

INPUT

No input is required because the grammar is predefined in the program.

OUTPUT



RESULT

Thus, the C program displays the LEADING sets for the given grammar used in an operator precedence parser.

EXPERIMENT – 18

AIM

To write a C program to compute the TRAILING sets for the given grammar used in an operator precedence parser.

ALGORITHM

1. Start the program.
2. Define the given grammar:
    - `E → E + T | T`
    - `T → T * F | F`
    - `F → ( E ) | id`
3. Compute the TRAILING set for each non-terminal.
4. For `E`, include `+`, `)`, and `id`.
5. For `T`, include `+`, `*`, `)`, and `id`.
6. For `F`, include `*`, `+`, `)`, and `id`.
7. Display the TRAILING sets.
8. Stop the program.

CODE

```
#include <stdio.h>

int main() {
    printf("GIVEN GRAMMAR:\n");
    printf("E -> E + T | T\n");
    printf("T -> T * F | F\n");
    printf("F -> ( E ) | id\n");

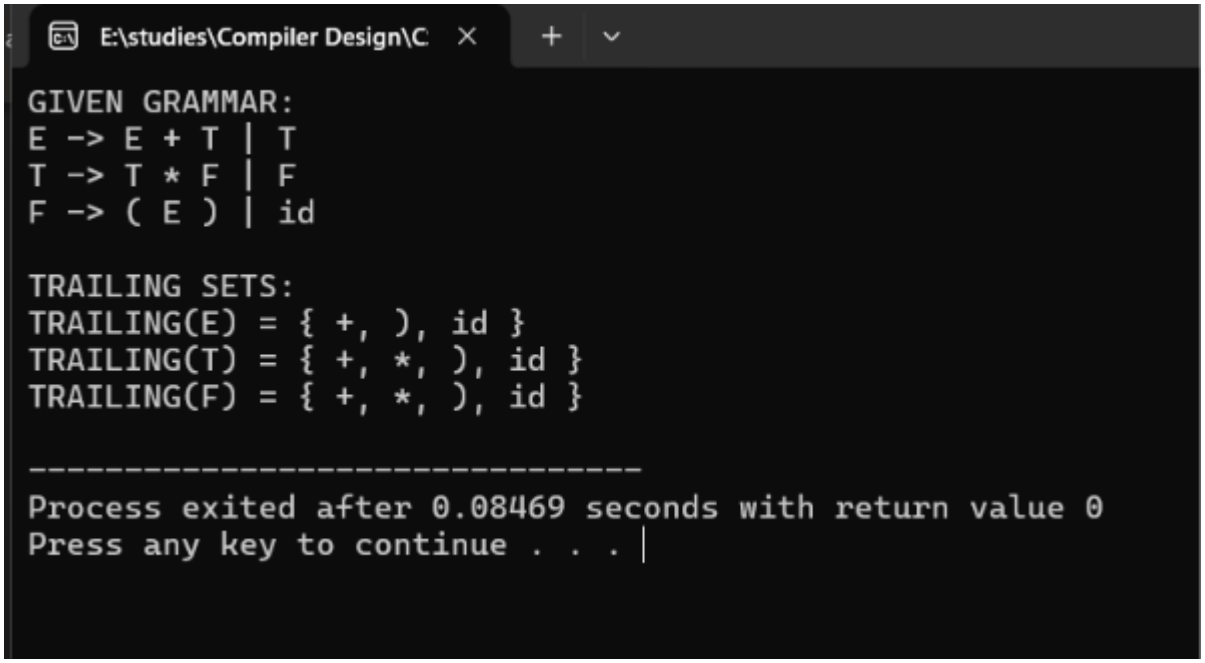
    printf("\nTRAILING SETS:\n");
    printf("TRAILING(E) = { +, ), id }\n");
    printf("TRAILING(T) = { +, *, ), id }\n");
    printf("TRAILING(F) = { +, *, ), id }\n");

    return 0;
}
```

INPUT

No input is required because the grammar is predefined in the program.

OUTPUT



RESULT

Thus, the C program displays the TRAILING sets for the given grammar used in an operator precedence parser.

EXPERIMENT – 19

AIM

To write a LEX specification that reads a C source program from a .c file, ignores redundant spaces, tabs, newlines and comments for word counting, and counts the total number of characters, words and lines in the source file.

ALGORITHM

1. Start the program.
2. Define counters for characters, words and lines.
3. Specify LEX rules to recognize whitespace, words, comments and other characters.
4. Read the C source code from stdinput.
5. Count characters, words and lines using the defined LEX rules.
6. Ignore comments while counting words, as well as redundant spaces and tabs as words.
7. Display the total number of characters, words and lines.
8. Stop the program.

CODE

```
%{

#include <stdio.h>

long characters = 0;

long words = 0;

long lines = 0;

int last_char = '\0';

%}


%option noyywrap

%x COMMENT


%%

"//"[^\n]* {
```

```
    characters += yyleng;
    last_char = yytext[yyleng - 1];
}

"/*" {
    characters += yyleng;
    last_char = '*';
    BEGIN(COMMENT);
}

<COMMENT>"*/" {
    characters += yyleng;
    last_char = '/';
    BEGIN(INITIAL);
}

<COMMENT>\n {
    characters++;
    lines++;
    last_char = '\n';
}

<COMMENT>. {
    characters++;
    last_char = yytext[0];
}

<COMMENT><<EOF>> {
```

```
    return 0;
}

\"([^\\\n]|\\.)*\" {
    characters += yyleng;
    words++;
    last_char = yytext[yyleng - 1];
}

\'([^\\\n]|\\.)*\' {
    characters += yyleng;
    words++;
    last_char = yytext[yyleng - 1];
}

\n {
    characters++;
    lines++;
    last_char = '\n';
}

[ \t\r]+ {
    characters += yyleng;
    last_char = yytext[yyleng - 1];
}

[A-Za-z_][A-Za-z0-9_]* {
    characters += yyleng;
```

```
    words++;
    last_char = yytext[yyleng - 1];
}

[0-9]+(\.[0-9]+)? {
    characters += yyleng;
    words++;
    last_char = yytext[yyleng - 1];
}

. {
    characters++;
    last_char = yytext[0];
}

%%

int main(void)
{
    yylex();

    if (characters > 0 && last_char != '\n')
        lines++;

    printf("\nNumber of characters = %ld\n", characters);
    printf("Number of words = %ld\n", words);
    printf("Number of lines = %ld\n", lines);
```

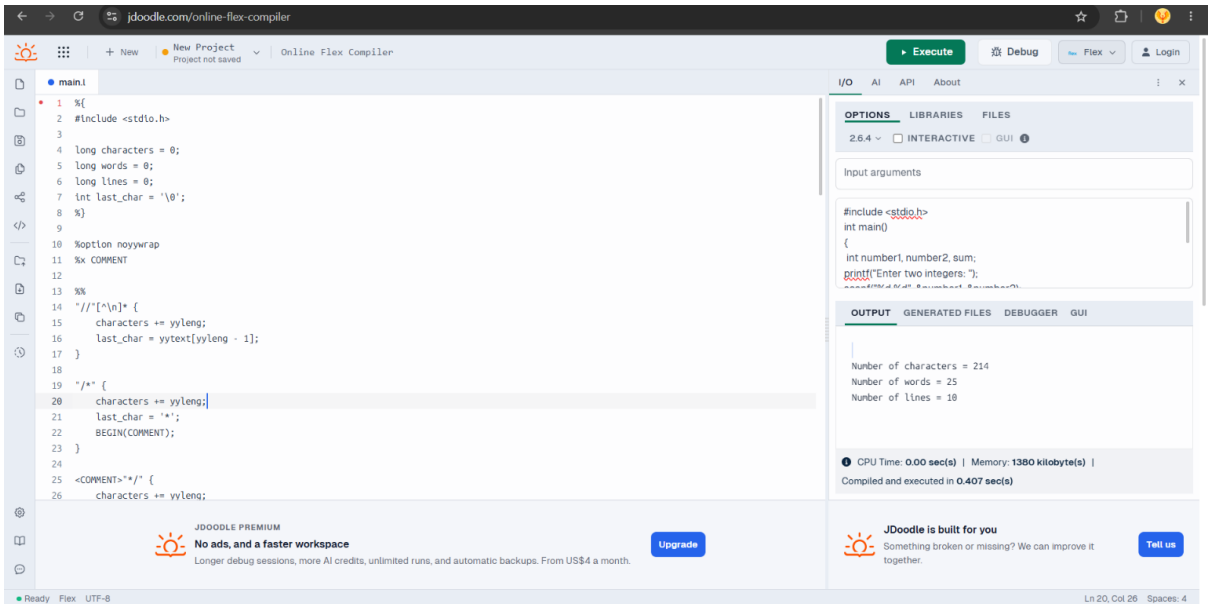
```
    return 0;

}
```

INPUT

```
#include <stdio.h>
int main()
{
    int number1, number2, sum;
    printf("Enter two integers: ");
    scanf("%d %d", &number1, &number2);
    sum = number1 + number2;
    printf("%d + %d = %d", number1, number2, sum);
    return 0;
}
```

OUTPUT



RESULT

Thus, the lex program ran successfully.

EXPERIMENT – 20

PROGRAM 1: LEX PROGRAM TO PRINT ALL CONSTANTS IN A C SOURCE PROGRAM

AIM

To write a LEX program to identify and print all constants present in a given C source program.

ALGORITHM

1. Start the program.
2. Read the C source code from the online compiler's input.
3. Define LEX rules to identify integer, floating-point, character, and string constants.
4. Print each constant encountered in the source code.
5. Ignore other tokens.
6. Stop the program after scanning the entire input.

CODE

```
%{
#include <stdio.h>
%}

%option noyywrap

%%
\"([^\\\"\n]|\\.)*\" {
    printf("String constant: %s\n", yytext);
}

\'([^\\'\n]|\\.)*\' {
    printf("Character constant: %s\n", yytext);
}

[0-9]+\.[0-9]+([eE][+-]?[0-9]+)? {
    printf("Floating-point constant: %s\n", yytext);
}

[0-9]+ {
    printf("Integer constant: %s\n", yytext);
}

[A-Za-z_][A-Za-z0-9_]* ;
[ \t\r\n]+ ;
. ;
%%

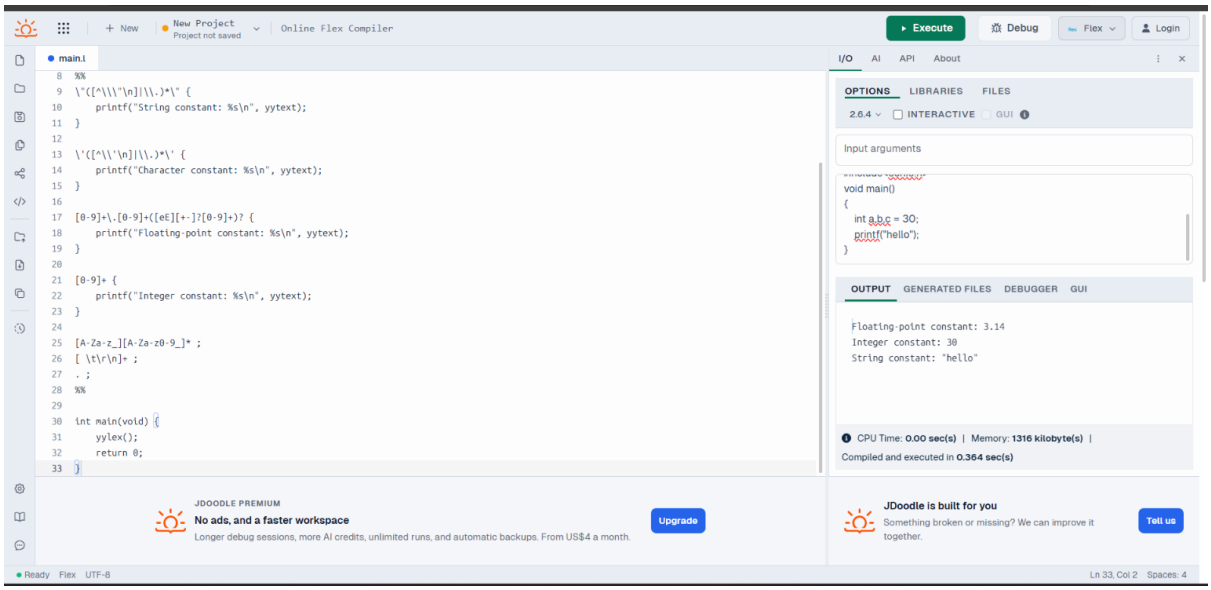
int main(void) {
    yylex();
    return 0;
```

```
}
```

INPUT

```
#define PI 3.14
#include<stdio.h>
#include<conio.h>
void main()
{
    int a,b,c = 30;
    printf("hello");
}
```

OUTPUT



RESULT

Thus, the LEX program identifies and prints integer, floating-point, character, and string constants in the given C source program.

PROGRAM 2: LEX PROGRAM TO COUNT MACROS AND HEADER FILES

AIM

To write a LEX program to count the number of macros defined and header files included in a given C source program.

ALGORITHM

1. Start the program.
2. Read the C source code from the online compiler's input.
3. Identify lines beginning with `#define` and increment the macro counter.
4. Identify lines beginning with `#include` and increment the header file counter.
5. Display the total number of macros and header files.
6. Stop the program.

CODE

```
%{
#include <stdio.h>

int macros = 0;
int headers = 0;
%}

%option noyywrap

%%
^[ \t]*#[ \t]*define[ \t]+[A-Za-z_][A-Za-z0-9_]* {
    macros++;
}

^[ \t]*#[ \t]*include[ \t]*[<\"][^>\"\n]+[>\"] {
    headers++;
}

\n ;
.  ;
%%

int main(void) {
    yylex();

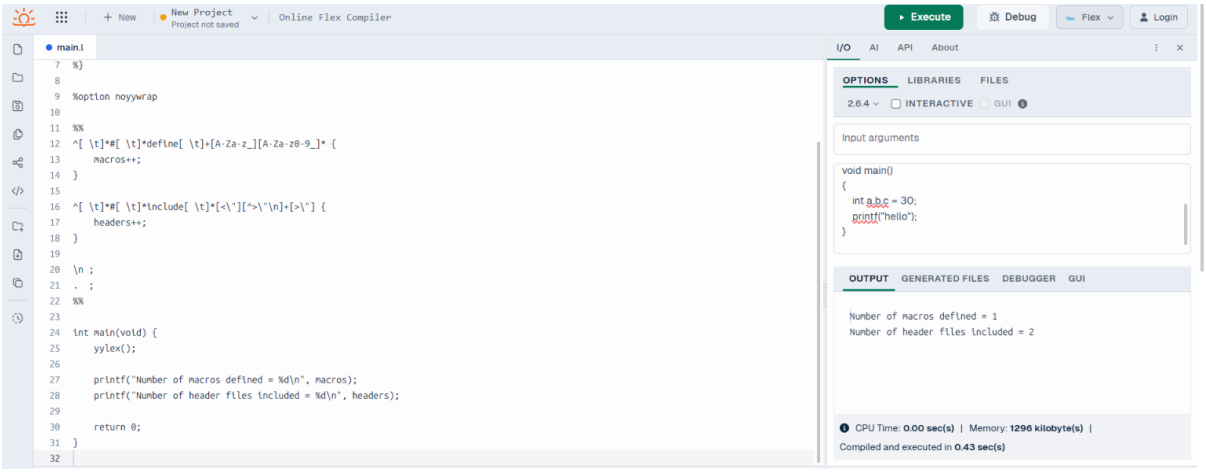
    printf("Number of macros defined = %d\n", macros);
    printf("Number of header files included = %d\n", headers);

    return 0;
}
```

INPUT

```
#define PI 3.14
#include<stdio.h>
#include<conio.h>
void main()
{
    int a,b,c = 30;
    printf("hello");
}
```

OUTPUT



RESULT

Thus, the LEX program successfully counts the macros defined and header files included in the given C source program.